

PL Final

AAM-Based Abstract Interpretation in Haskell

Gerald Hoff

Electrical Engineering and Computer Science

Washington State University

Pullman, WA

gerald.hoff@wsu.edu

I. AAM IMPLEMENTATION

Overview

Abstracting Abstract Machines derives a static analysis from a concrete machine with two main ideas: bound the address space so the store stays finite, and store sets of values instead of single values so that merged addresses do not lose information. The result is a nondeterministic transition relation whose reachable states over-approximate every possible concrete execution.

This implementation (github) runs a CESK* machine over the pure λ -calculus. The object language has variables, lambdas, and applications, all encoded as a Haskell ADT with constructors `Ref`, `Lam`, and `(:@)`.

Machine State

A state is a five-tuple $\langle e, \rho, \hat{\sigma}, \kappa, t \rangle$: the control expression e , environment $\rho : \text{Var} \rightarrow \text{Addr}$, abstract store $\hat{\sigma} : \text{Addr} \rightarrow \mathcal{P}(\text{Storable})$, current continuation κ , and time stamp t . For CESK* continuations are *store-allocated*: each frame is stored at a `KAddr` address rather than kept on an explicit stack. This ensures that the stack never grows outside the store, so the analysis is finite.

Time and addresses. Time is a k -length call-string built from expressions. `tick` advances it by prepending the current expression and truncating:

```
tick :: Sigma -> Time
tick (e, _, _, _, t) = take k (e : t)
```

Binding addresses are `BAddr(x, t)` and continuation addresses are `KAddr(e, t)`. At $k = 0$ every call maps to the same address (0-CFA monovariance); at $k = 1$ addresses distinguish the most recent call site (1-CFA sensitivity).

Abstract store. Because addresses are finite, two different bindings may land at the same address. The store maps addresses to *sets* and uses a join-insert rather than overwrite:

```
(\!)! :: (Ord k, Lattice v)
=> (k -> v) -> [(k, v)] -> (k -> v)
f \! [(key, v)] = Map.insertWith (join) key v f
```

Lookup returns the empty set for missing keys, so the machine never crashes on unknown addresses.

Transition Rules

`step :: Sigma -> [Sigma]` returns a list of successors; multiple results arise when an address holds more than one value.

Variable lookup iterates over every closure at the variable's address, producing one successor per closure:

```
step state@(Ref x, rho, sigma, kappa, t) =
  [ (Lam lam, rho', sigma, kappa, t')
  | Clo (lam, rho') <-
    Set.toList (sigma !! (rho ! x)) ]
  where t' = tick state
```

Application allocates a fresh `KAddr`, saves the current continuation there, and pushes an `Ar` frame so the machine next evaluates the operator:

```
step state@(f :@ e, rho, sigma, kappa, t) =
  [ (f, rho, sigma', kappa', t') ]
  where
    t' = tick state
    a' = allocKont (f :@ e) t'
    sigma' = sigma oplus! [a' ==> s (Cont kappa)]
    kappa' = Ar (e, rho, a')
```

Call fires when both sides are evaluated (the `Fn` frame). A fresh `BAddr` is allocated for the parameter, the argument closure is joined into the store, and the saved continuation is retrieved.

```
step state@(Lam lam, rho, sigma,
  Fn (x ==> e, rho', a), t) =
  [ (e, rho' // [x ==> a'],
    sigma oplus! [a' ==> s (Clo (lam, rho))],
    kappa, t')
  | Cont kappa <- Set.toList (sigma !! a) ]
  where t' = tick state
        a' = allocBind x t'
```

Exploration and Tests

A worklist search collects all reachable states, skipping any already seen. Termination is guaranteed because the state space is finite. Final states match $(\text{Lam } _, _, _, \text{Mt}, _)$.

Four programs check correctness: identity applied to a lambda (`test1`), a higher-order function (`test2`), Church successor applied to Church zero (`test3`), and Ω . The latter should of course cycle through states without ever producing a final state, confirming the analysis terminates even on non-terminating programs.

II. COMPARISON WITH SUMMARY BASED PROGRAM ANALYSIS

The implementation of Abstracting Abstract Machines (AAM) described in this project provides a systematic way to derive static analyses from concrete operational semantics. This approach is in contrast to the summary-based interprocedural dataflow analysis framework which transforms dataflow problems into graph-reachability problems.

Methodological Foundations

AAM is fundamentally a derivational approach that begins with a concrete abstract machine, such as the CESK machine, and refactors it into a finite-state version by bounding the address space of the store. It relies on store-allocated continuations to eliminate recursive structures in the state-space, ensuring that the resulting abstract transition relation is finite and computable. In contrast, the summary-based approach described by Reps, Horwitz, and Sagiv focuses on solving a specific class of dataflow-analysis problems by converting them into a realizable-path reachability problem on an exploded supergraph. While AAM simulates the execution of a machine, the IFDS framework treats the program as a collection of procedure flow graphs and uses a tabulation algorithm to determine if a dataflow fact at one point can reach another point via a path that respects the call-return structure of the program.

Interprocedural Precision and the Stack

Both approaches aim for "precise" interprocedural analysis, but they handle the program stack differently:

- **AAM Strategy:** AAM achieves finiteness by allocating continuations (the stack) in the store. This allows the stack to be abstracted using the same mechanisms as variable bindings, where multiple continuation frames can be joined at a single abstract address. Precision is tuned through the use of time-stamps or call-strings, where a k -length string distinguishes different calling contexts (k -CFA).
- **Summary-Based Strategy:** The IFDS framework achieves precision by ensuring reachability only along "valid paths," where "returns" are matched with their corresponding "calls" using a context-free grammar of balanced parentheses. It utilizes "summary edges" at call sites to record the effects of a procedure call, allowing the analysis to avoid re-analyzing a procedure's body for every distinct call site.

Efficiency and Complexity

The IFDS framework offers strong theoretical guarantees on complexity. Specifically, it can solve distributive interprocedural problems in polynomial time, $O(ED^3)$, where E is the number of edges in the supergraph and D is the size of the dataflow fact set. This efficiency is made possible by the restriction that dataflow functions must distribute over the confluence operator (union or intersection). AAM is generally more computationally expensive because it explores a state-space that

tracks the history of the machine's execution. This produces more intensional view-tracking of variables at the cost of the complexity. However, AAM can be widened to a single-threaded global store to achieve polynomial-time performance similar to traditional k -CFA.

Generality and Expressiveness

A significant advantage of the AAM approach is its extreme generality. Because it is derived directly from the operational semantics, it scales naturally to handle complex language features such as first-class continuations, by-need evaluation, and garbage collection. For instance, AAM can soundly approximate the Ω -combinator or programs with side effects simply by applying the same structural abstraction to the relevant machine rules. The summary-based IFDS framework is more specialized. It is highly effective for "locally separable" or "distributive" problems (such as reaching definitions or live variables) but requires significant adaptation for problems that do not fit the distributive subset model. While the IDE (Interprocedural Distributive Environment) framework generalizes IFDS to handle some non-subset problems, it still relies on a distributive structure that may not easily capture the full range of intensional properties that an AAM-based machine can explore.

III. ACCELERATE TO A FIXPOINT

AAM allows for the finitizing of recursive code. This is a core limitation of the Accelerate DSL for writing high performance Haskell code. It can compile Haskell code down to CUDA only if the code is not recursive. By applying this analytical strategy from AAM we can theoretically allow for efficient Haskell code that iterates to a fixpoint.

An implementation that allows for any type of recursion would be an immense undertaking. As such, we implement this idea for a small subset of recursive programs to validate the concept. The easiest application would be to iterative programs where the array size is invariant. Standard Accelerate would require such programs to be externally relaunched. By determining that a program has an invariant array size we can instead have the code stay on the GPU, saving time that would be spent relaunching the kernel. Additional analysis can of course be derived using AAM, and further optimizations can be made in theory. But for the purpose of demonstrating the power of combining these approaches we will stick with this problem subset.

Static Analysis of the Laplace Solver

This project, located here, demonstrates this methodology through several program examples, including a Laplace equation solver. This is a classic iterative stencil computation. In this implementation, the Laplace program is represented as a series of Jacobi iterations where each cell in a 2D grid is updated based on its neighbors until the system reaches a steady state. The AAM-based static analyzer processes the Accelerate code by tracking the abstract state of the grid's dimensions across recursive calls. Because the analyzer uses a finite approximation of the store, it can symbolically execute

the iteration loop and observe that the stencil operations (like stencil or map) preserve the array's "shape" and memory footprint. By verifying this shape invariance statically, the analyzer confirms that the recursion does not lead to unbounded memory allocation, proving that the loop can be safely unrolled or managed within a persistent GPU kernel.

Once static analysis confirms the invariance of the Laplace solver, the compilation pipeline uses this information to optimize the generated code. Instead of returning control to the CPU after each iteration we relaunch the kernel. This allows for persistent memory, kernel fusion, and reduced overheads. This AAM driven analysis allows for a more expressive version of the Accelerate DSL.

REFERENCES

- [1] D. Van Horn and M. Might, "Systematic Abstraction of Abstract Machines," *J. Functional Programming*, vol. 22, no. 4–5, pp. 705–746, 2012.
- [2] D. Van Horn and M. Might, "Abstracting Abstract Machines," *Proc. 15th ACM SIGPLAN Int. Conf. on Functional Programming (ICFP '10)*, pp. 101–112, 2010.
- [3] T. Reps, S. Horwitz, and M. Sagiv, "Precise Interprocedural Dataflow Analysis using Graph Reachability," *Proc. 22nd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL '95)*, pp. 49–61, 1995.
- [4] T. L. McDonell, M. M. T. Chakravarty, G. Keller, and B. Lippmeier, "Optimising Purely Functional GPU Programs," *ACM SIGPLAN Notices*, vol. 48, no. 9, pp. 49–60, 2013.